# AI-Powered Resume Fit Analyzer — Full Project Spec

**Type:** TY (Third Year) Computer Engineering mini project, group of 2–3
**Team:** [Names] · **Guide:** [Guide name]
**One-line pitch:** A web app that compares a PDF resume with a job description and tells a job seeker, *before they apply*: their fit score, which skills they are missing, what to learn (with free links), and whether the resume format will survive ATS (Applicant Tracking System) screening.

---

## 0. How to use this file

This file is the single source of truth for building the real app with a coding agent (Claude Code, Antigravity, etc.).

- **Claude Code:** save it in the repo root as `CLAUDE.md` (or keep `PROJECT_SPEC.md` and tell it "read PROJECT_SPEC.md first").
- **Antigravity / other agents:** attach it in the first prompt and say "Read PROJECT_SPEC.md fully, then follow Section 17 (build order) one step at a time."
- Part A (Sections 1–7) is the content of both review presentations (Phase 1 pitch, Phase 2 and Phase 3 reviews). Part B (Sections 8–20) is what to build and how.
- Where the two presentations differ, **the final Phase 2 & 3 presentation wins** (it added sentence embeddings, measurable objectives and a timeline).

### Instructions for the coding agent

1. Read the whole file before writing code. Build in the order of Section 17 and stop after each step to run the acceptance checks.
2. Keep the five NLP modules (Parse, Skills, Match, Gap, ATS) as **separate files with a clear function interface**. The project is judged on its design, so do not collapse everything into `main.py`.
3. Do not invent data. Learning-resource URLs must be real; run the link checker in Section 10.4. If you are not sure a URL exists, leave it out.
4. Never store uploaded resumes or job descriptions, and never log their content (Section 16).
5. Use plain JavaScript (JSX) on the frontend and Python 3.11+ on the backend. Add type hints and docstrings in Python. Add a unit test for every module.
6. Everything must run on a normal laptop with free tools. No paid APIs, no GPU, no training.
7. When a requirement says "starting value", put it in `config.py` so the team can tune it later.
8. **The tech stack is locked to what the review slides show (Section 7).** Core tools: React.js, Chart.js, Python, FastAPI, pdfplumber, spaCy, NLTK, scikit-learn, sentence-transformers, SQLite, Render, Vercel, Git + GitHub. Small supporting libraries (Vite, react-router-dom, axios, react-chartjs-2, numpy, joblib, pytest, etc.) are fine. Do **not** replace or add a core tool (another database, another NLP or embedding library, another frontend framework, another host) without asking the team first, because the slides and the app must match.

---

# PART A — PROJECT CONTENT (from the presentations)

## 1. Title and problem statement (Phase 1 presentation)

**Title:** AI-Powered Resume Fit Analyzer
**Subtitle:** Scoring resume–job fit and mapping the skill gap
**Label on slide:** Mini Project · TY

Problem statement, as on the Phase 1 slide:

- Job seekers can't tell if they're a good fit before applying.
- When rejected, they don't know which skills they were missing.
- Recruiters waste time screening resumes manually.
- Resumes get auto-rejected by ATS for formatting issues.

## 2. Purpose

Show a job seeker, before applying, their **fit score, missing skills and what to learn**.

## 3. Scope (Phase 1 presentation)

As on the Phase 1 slide:

- Take a resume (PDF) and a job description as input.
- Score how well they match.
- List the missing skills and suggest what to learn.
- Check the resume for ATS-friendly formatting.
- Show the result as a score and a simple chart.

Added by the Phase 2 and 3 presentation: compare two matching methods (TF-IDF baseline vs sentence embeddings) on hand-labelled resume–job pairs.

**Out of scope for this build (team's suggestions, not on the slides):** multi-resume recruiter ranking, LLM-based rewrite suggestions, multi-language resumes, browser extension, user accounts and scan history, OCR for scanned PDFs, DOCX upload.

### 3.1 Proposed solution (Phase 1 presentation)

Five parts. These are the same five parts that became the NLP engine modules in Part B.

| Part (slide name) | What it does | Module in Part B |
|---|---|---|
| Parser | Reads text from the resume and job description. | `parser.py` |
| Matching Engine | TF-IDF + cosine similarity give a fit score. | `matcher_tfidf.py`, later `matcher_embed.py` |

| Part (slide name) | What it does | Module in Part B |
|---|---|---|
| Skill-Gap Roadmap | Finds missing skills and links resources to learn them. | `skills.py` + `gap.py` |
| ATS Checker | Flags formatting that ATS software rejects. | `ats.py` |
| Visualizer | Shows the fit as a score and chart. | Frontend charts |

### 3.2 Workflow (Phase 1 presentation)

1. **Upload Resume & Job Description** — user uploads a PDF resume and pastes the job description into the web form (React front-end).
2. **Extract & Clean Text** — pdfplumber pulls raw text from the PDF; the text is lowercased and stripped of stopwords and symbols.
3. **Identify Skills & Keywords** — spaCy / NLTK extract skills and key terms from both the resume and the job description.
4. **Convert Text to Vectors** — scikit-learn TF-IDF turns the cleaned text into numeric vectors the model can compare.
5. **Compute Match Score** — cosine similarity between the two vectors produces a percentage fit score.
6. **Analyze Gap & Show Results** — missing skills are mapped to learning resources; score, chart and ATS flags are returned via FastAPI.

### 3.3 Technical stack (Phase 1 presentation)

Frontend: React.js, Chart.js · Backend: Python, FastAPI · ML / NLP: scikit-learn, spaCy, pdfplumber · Database: SQLite · Deployment: Render / Vercel (free tier) · Tools: Git + GitHub. The final presentation adds NLTK, sentence-transformers and the "why it fits" column (Section 7); nothing from Phase 1 was removed.

## 4. Literature review (Phase 2)

Numbers below are quoted from the team's final presentation. The team should re-check them against the papers before the final report.

| Paper | What they did | Limitation | How our project differs |
|---|---|---|---|
| Kaur, Gill & Gill (2025), *Resume Sorting Using ML – A Review* | Reviewed ~20 systems (SVM, KNN, BERT). Best ≈ 96% using TF-IDF features. | Only helps recruiters sort resumes. No feedback for the candidate. | Tells the job seeker the missing skills and what to learn. |
| Lavi et al. (2021), *conSultantBERT – RecSys in HR* | Fine-tuned SBERT on 274k labelled pairs. Beat TF-IDF (0.85 vs 0.57 AUC). | Needs huge private data and GPU. Gives a score with no reason. | Ready-made model, no training. Explains the score. |

| Paper | What they did | Limitation | How our project differs |
|---|---|---|---|
| Grech & Suda (2020), *Neural Résumé Search – ICPRAM* | Word embeddings + TF-IDF + cosine on 2,000 resumes. | Tested on only 3 jobs. Ignores word order and context. | Context-aware embeddings + our own labelled test set. |

Links found so far:

- conSultantBERT: https://arxiv.org/pdf/2109.06501
- Neural IR for résumé searching (SciTePress, 2020): https://www.scitepress.org/Papers/2020/93550
- Algorithmic friction in automated resume screening (arXiv 2026, background on ATS problems): https://arxiv.org/html/2602.04087v1

## 5. Justification of the project (Phase 2)

- **Gap:** most work reviewed ranks resumes for recruiters. The job seeker is not told why they don't fit.
- Job seekers need feedback before applying, so we give missing skills and a learning plan.
- TF-IDF alone is weak (0.57 AUC in the literature), so we add sentence embeddings and compare both.
- Big models need private data and GPUs. Ours uses free tools and a normal laptop.
- Resumes get rejected by ATS for formatting, so we flag those issues too.

## 6. Objectives, abstract, methodology (Phase 3)

### 6.1 Objectives (SMART)

| # | Objective | Measure | By |
|---|---|---|---|
| 1 | Extract text from PDF resume and job description | 9 of 10 test PDFs read correctly | Week 3 |
| 2 | Give a fit score (TF-IDF + cosine baseline) | Score for 10+ hand-labelled pairs | Week 5 |
| 3 | Add sentence embeddings and compare | ≥ 70% agreement with hand labels | Week 7 |
| 4 | List missing skills with free learning links | 100+ skills, 1+ link each | Week 8 |
| 5 | Check ATS formatting and show results in a web app | 8+ checks; result in ≤ 5 seconds | Week 11 |

### 6.2 Abstract

A web app that compares a PDF resume with a job description. It gives a fit score, the missing skills, a learning plan and ATS warnings. Matching uses TF-IDF + cosine similarity as a baseline, upgraded to sentence embeddings, and both are tested on hand-labelled resume–job pairs.

### 6.3 Methodology — iterative and incremental

Cycle: **Study → Design → Build → Test → Review**, repeated for each part.

1. Study: needs + papers
2. Design: architecture
3. Build: one part at a time
4. Test: on labelled pairs
5. Review: with guide

Why this method:

- A working version early (the TF-IDF score), then improve step by step.
- Every upgrade is tested on the same data, so gains are proven.
- Fits the phase reviews; problems are caught early, not at the end.

## 7. Design and technology stack (final)

All open-source, current and widely used in industry.

| Layer | Tool | Why it fits |
|---|---|---|
| Frontend | React.js, Chart.js | Upload form and score charts; most used UI library |
| Backend | Python, FastAPI | Same language as the ML code; fast and modern |
| NLP | pdfplumber, spaCy, NLTK | Read PDF text and find skills; industry standard |
| Matching | scikit-learn, sentence-transformers | TF-IDF baseline + meaning-based embeddings |
| Database | SQLite | Stores skills and learning links; no server needed |
| Deploy & tools | Render, Vercel, Git + GitHub | Free hosting and team version control |

Each module solves one problem point and one objective.

### 7.1 TF-IDF vs sentence embeddings (what is being compared)

- **TF-IDF** turns text into numbers by counting words, weighted by how special each word is. It only matches exact words, so "built websites using React" and "frontend developer experience" score low even though they mean the same.
- **Sentence embeddings** (a ready-made pre-trained model, no training) turn a sentence into numbers that capture its *meaning*, so those two sentences score high.
- Both end with the same step: **cosine similarity** between resume numbers and job-description numbers, giving a match percentage.
- Comparison: run both on the same 10+ hand-labelled resume–JD pairs and see which agrees more with the hand labels (target for embeddings: ≥ 70%).

---

# PART B — WHAT TO BUILD

## 8. System architecture

```
flowchart LR
  U[User - browser] --> FE[React + Chart.js]
  FE -- "POST /api/analyze (PDF + JD)" --> API[FastAPI]
  API --> V[Validation]
  V --> O[Orchestrator]
  O --> P[Parse and clean]
  O --> A[ATS checker]
  P --> S[Skills extractor]
  P --> M[Matcher: TF-IDF and embeddings]
  S --> G[Gap analyzer]
  DB[(SQLite: skills, learning_links)] --> S
  DB --> G
  M --> R[Response builder]
  G --> R
  A --> R
  R -- JSON --> FE
```

A rendered diagram is in `docs/architecture.png` / `docs/architecture.svg` (same content, with the request flow written below it).

**Layers**

1. **Client:** React web form, results view, Chart.js charts.
2. **API:** FastAPI endpoints, request validation, orchestrator, JSON response builder.
3. **Processing:** the five NLP modules (Parse, Skills, Match, Gap, ATS).
4. **Storage:** SQLite with two tables, `skills` and `learning_links`. Nothing about the user is stored.

**Request flow**

1. User uploads a PDF resume and pastes the job description.
2. React sends both to `POST /api/analyze` using Axios.
3. FastAPI validates the input; the orchestrator calls the NLP engine.
4. Parse: pdfplumber extracts text; it is cleaned.
5. Skills: spaCy finds skills in both texts using the skills list from SQLite.
6. Match: TF-IDF + cosine (baseline) and sentence embeddings + cosine give the fit scores.
7. Gap: JD skills minus resume skills; learning links are looked up in SQLite.
8. ATS: 8+ rule-based formatting checks run on the resume PDF.
9. Response builder returns JSON; React shows score, charts, missing skills and ATS flags.

**Offline (not part of the live request):** the evaluation script runs both matchers on the labelled pairs and writes a report that the web app shows on the Evaluation page.

## 9. Repository layout

```
resume-fit-analyzer/
├── PROJECT_SPEC.md             # this file (or CLAUDE.md)
```

```
├── README.md
├── docs/
│   ├── architecture.png
│   └── architecture.svg
├── backend/
│   ├── requirements.txt
│   ├── app/
│   │   ├── main.py              # FastAPI app, CORS, startup (load models once)
│   │   ├── config.py            # all tunable constants + env vars
│   │   ├── schemas.py           # Pydantic request/response models
│   │   ├── db.py                # SQLite connection + queries
│   │   ├── routers/
│   │   │   ├── analyze.py       # POST /api/analyze
│   │   │   ├── skills.py        # GET /api/skills
│   │   │   ├── evaluation.py    # GET /api/evaluation
│   │   │   └── health.py        # GET /api/health
│   │   └── services/
│   │       ├── parser.py        # PDF -> text, cleaning
│   │       ├── skills.py        # skill extraction + JD importance
│   │       ├── matcher_tfidf.py # baseline
│   │       ├── matcher_embed.py # sentence embeddings
│   │       ├── gap.py           # missing skills + roadmap
│   │       ├── ats.py           # ATS checks
│   │       └── pipeline.py      # orchestrator
│   ├── data/
│   │   ├── skills_seed.json     # 100+ skills with aliases + links
│   │   └── app.db               # generated by scripts/seed_db.py
│   ├── scripts/
│   │   ├── seed_db.py
│   │   └── check_links.py
│   ├── eval/
│   │   ├── pairs.json           # hand-labelled pairs (Section 12)
│   │   ├── resumes/  jds/
│   │   ├── evaluate.py
│   │   └── report.json          # output, served by /api/evaluation
│   └── tests/
│       ├── fixtures/            # test PDFs (Section 14)
│       └── test_*.py
└── frontend/
    ├── package.json  vite.config.js  index.html
    └── src/
        ├── main.jsx  App.jsx  api.js
        ├── pages/ (Analyzer, HowItWorks, Evaluation, About)
        ├── components/ (see Section 13)
        ├── data/ (sampleJobs.js)
        └── styles/ (tokens.css, global.css)
```

## 10. Backend specification

### 10.1 Dependencies

`fastapi`, `uvicorn[standard]`, `python-multipart`, `pdfplumber`, `spacy` (+ `en_core_web_sm`), `nltk` (stopwords), `scikit-learn`, `sentence-transformers`, `numpy`, `pydantic`, `joblib`, `pytest`, `httpx`. One-time

setup: `python -m spacy download en_core_web_sm` and `nltk.download("stopwords")`.

### 10.2 Configuration (`config.py`, all overridable by env vars)

| Name | Default | Meaning |
|---|---|---|
| `MAX_UPLOAD_MB` | 5 | Max PDF size |
| `MAX_PDF_PAGES` | 5 | Reject longer PDFs |
| `MIN_JD_CHARS` / `MAX_JD_CHARS` | 50 / 10000 | JD length limits |
| `EMBEDDING_MODEL` | `sentence-transformers/all-MiniLM-L6-v2` | Small, CPU-friendly (384 dimensions) |
| `PRIMARY_METHOD` | `embeddings` | Which score is shown as the main fit score (`tfidf` or `embeddings`) |
| `TFIDF_CEILING` | 0.60 | Raw cosine mapped to 100% (starting value) |
| `EMBED_CEILING` | 0.80 | Same, for embeddings (starting value) |
| `LABEL_GOOD_MIN` / `LABEL_AVG_MIN` | 65 / 40 | Score thresholds for "Good / Average / Low fit" (starting values) |
| `ALLOWED_ORIGINS` | `http://localhost:5173` | CORS |
| `DB_PATH` | `data/app.db` | SQLite file |

### 10.3 API

All routes are under `/api`. Errors always use `{"error": {"code": "...", "message": "..."}}`.

`POST /api/analyze` — `multipart/form-data`

- `resume` (file, PDF, ≤ 5 MB)
- `job_description` (text, 50–10,000 characters)
- `method` (optional: `embeddings`, `tfidf`, `both`; default `both`)

Success `200`:

```
{
  "request_id": "b1f0...",
  "processing_ms": 2310,
  "fit": {
    "primary_method": "embeddings",
    "score": 72.4,
    "label": "Good fit",
    "tfidf_score": 41.2,
```

```
    "embedding_score": 72.4
  },
  "skills": {
    "resume": ["python", "react", "sql"],
    "matched": [{"name": "python", "category": "Programming Languages"}],
    "missing": [
      {
        "name": "docker",
        "display_name": "Docker",
        "category": "Cloud & DevOps",
        "importance": "high",
        "mentions": 3,
        "resources": [
          {"title": "Docker: Get started", "url": "https://docs.docker.com/get-started/",
           "provider": "Docker Docs", "resource_type": "docs", "level": "beginner", "is_free":
true}
        ]
      }
    ],
    "coverage_percent": 62.5,
    "by_category": [{"category": "Web Backend & APIs", "required": 4, "matched": 3}]
  },
  "explanation": {
    "strong_matches": [{"jd_snippet": "...", "resume_snippet": "...", "similarity": 0.81}],
    "weak_spots":     [{"jd_snippet": "...", "best_resume_snippet": "...", "similarity": 0.22}]
  },
  "ats": {
    "score": 75,
    "passed": 7,
    "total": 10,
    "checks": [
      {"id": "multi_column", "title": "Single-column layout", "status": "pass|warn|fail",
       "detail": "Text flows in one column.", "fix": null}
    ]
  },
  "warnings": ["The job description is very short, so the score may be unreliable."]
}
```


Errors:

| HTTP | code | When |
|---|---|---|
| 400 | `invalid_file_type` | Not a real PDF (check the `%PDF` magic bytes, not just the extension) |
| 413 | `file_too_large` | Over `MAX_UPLOAD_MB` |
| 422 | `jd_too_short` / `jd_too_long` | JD length out of range |
| 422 | `unreadable_pdf` | No extractable text (probably a scanned image; OCR is out of scope) |
| 422 | `too_many_pages` | Over `MAX_PDF_PAGES` |
| 500 | `internal_error` | Anything else (message must not leak internals) |

`GET /api/health` → `{"status": "ok", "model_loaded": true, "version": "1.0.0"}`. Used for the demo warm-up (Section 18).
`GET /api/skills?query=&category=` → list of skills with resources (for a skills browser, optional).
`GET /api/evaluation` → contents of `eval/report.json` (Section 12), or `404` with `{"error": {"code": "no_report"}}` if not generated yet.

### 10.4 Database ( `SQLite` )

Only these two tables. No resumes, no job descriptions, no users.

```
CREATE TABLE skills (
  id            INTEGER PRIMARY KEY,
  name          TEXT NOT NULL UNIQUE,      -- canonical, lowercase, e.g. 'node.js'
  display_name  TEXT NOT NULL,             -- e.g. 'Node.js'
  category      TEXT NOT NULL,
  aliases       TEXT NOT NULL DEFAULT '[]' -- JSON array, e.g. ["nodejs","node js"]
);
CREATE TABLE learning_links (
  id            INTEGER PRIMARY KEY,
  skill_id      INTEGER NOT NULL REFERENCES skills(id),
  title         TEXT NOT NULL,
  url           TEXT NOT NULL,
  provider      TEXT NOT NULL,
  resource_type TEXT NOT NULL CHECK (resource_type IN
('docs','course','video','tutorial','book')),
  level         TEXT NOT NULL CHECK (level IN ('beginner','intermediate','advanced')),
  is_free       INTEGER NOT NULL DEFAULT 1,
  UNIQUE (skill_id, url)
);
```

**Seed file** `data/skills_seed.json` (loaded by `scripts/seed_db.py`, which is idempotent):

```
[
  {"name": "python", "display_name": "Python", "category": "Programming Languages",
   "aliases": ["python3"],
   "links": [{"title": "The Python Tutorial", "url": "https://docs.python.org/3/tutorial/",
              "provider": "Python Docs", "resource_type": "docs", "level": "beginner"}]}
]
```

Requirements (Objective 4): **100+ skills, each with at least 1 free link.** Starter list by category (add aliases for each):

- **Programming Languages:** Python, Java, JavaScript, TypeScript, C, C++, C#, Go, Rust, Kotlin, Swift, PHP, Ruby, R, SQL, Bash, MATLAB
- **Web Frontend:** HTML, CSS, React, Next.js, Vue.js, Angular, Redux, Tailwind CSS, Bootstrap, Sass, Webpack, Vite, jQuery, Responsive Design, Accessibility
- **Web Backend & APIs:** Node.js, Express.js, FastAPI, Flask, Django, Spring Boot, ASP.NET, REST API, GraphQL, gRPC, WebSockets, Microservices, Authentication, JWT, OAuth
- **Databases:** MySQL, PostgreSQL, SQLite, MongoDB, Redis, Firebase, Oracle, DynamoDB, Elasticsearch, Database Design, Normalization

- **Cloud & DevOps:** Docker, Kubernetes, AWS, Azure, Google Cloud, CI/CD, GitHub Actions, Jenkins, Terraform, Ansible, Linux, Nginx, Serverless, Monitoring
- **Data Science & ML:** Machine Learning, Deep Learning, NLP, Computer Vision, scikit-learn, TensorFlow, PyTorch, Keras, Pandas, NumPy, Matplotlib, Data Analysis, Data Visualization, Statistics, Power BI, Tableau, Excel, Hugging Face, OpenCV, LLMs, Prompt Engineering, Spark, Hadoop, ETL
- **Core CS:** Data Structures, Algorithms, OOP, Operating Systems, DBMS, Computer Networks, System Design, Design Patterns, Concurrency, Compiler Design
- **Tools & Practices:** Git, GitHub, Jira, Agile, Scrum, Postman, VS Code, Unit Testing, Code Review, Debugging, Technical Writing
- **Testing & QA:** Selenium, Jest, PyTest, JUnit, Cypress, Test Automation, Manual Testing
- **Mobile:** Android, iOS, Flutter, React Native
- **Security:** Cybersecurity, Cryptography, Network Security, Penetration Testing, OWASP

Example links known to exist (still run the checker): Python `https://docs.python.org/3/tutorial/`, JavaScript `https://javascript.info/`, React `https://react.dev/learn`, FastAPI `https://fastapi.tiangolo.com/tutorial/`, Docker `https://docs.docker.com/get-started/`, Git `https://git-scm.com/book/en/v2`, scikit-learn `https://scikit-learn.org/stable/user_guide.html`, Machine Learning `https://developers.google.com/machine-learning/crash-course`, Kubernetes `https://kubernetes.io/docs/tutorials/kubernetes-basics/`, Node.js `https://nodejs.org/en/learn`, TypeScript `https://www.typescriptlang.org/docs/handbook/intro.html`.

`scripts/check_links.py` sends a request to every URL, prints any that don't return 200, and exits non-zero if there are failures. Run it before every release. Prefer official docs, freeCodeCamp, MDN, Kaggle Learn, university courses and well-known YouTube channels.

### 10.5 NLP pipeline

Each service module exposes one main function and no web code, so it can be tested alone.

#### 10.5.1 `parser.py` — Objective 1

- `extract_text(pdf_bytes) -> ParsedResume` where `ParsedResume` has `text`, `pages`, `lines` (with positions, reused by the ATS module) and `fonts`.
- Use `pdfplumber`. Handle two-column resumes by reading words and grouping by column (x-position clusters) before joining, so columns don't interleave.
- Remove repeated headers/footers and page numbers.
- If fewer than ~200 characters are extracted, raise `UnreadablePdf`.
- `clean_text(text) -> str`: lowercase, remove URLs, emails and phone numbers, normalise bullets and whitespace, keep tokens such as `c++`, `c#`, `node.js`. Keep a second, lightly cleaned copy (original casing, sentences intact) for sentence-based embeddings and for showing snippets.
- **Bias safety:** strip name-like header lines, emails, phone numbers, URLs and street addresses before scoring. Do not infer or use gender, age, nationality or photos.
- JD text goes through the same `clean_text`.

#### 10.5.2 `skills.py`

- At startup load all skills and aliases from SQLite and build a spaCy `PhraseMatcher` (match on lowercase). Multi-word skills ("machine learning", "rest api") must work.
- Normalise aliases to the canonical name (`js` → `javascript`, `reactjs` → `react`, `postgres` → `postgresql`, `k8s` → `kubernetes`).
- Ambiguous names (`c`, `r`, `go`) need extra care: match only case-sensitively as a standalone token and only when it appears in a skills/tech context (inside a list of technologies or next to other tech terms).
- `extract_skills(text) -> list[SkillHit]` returns canonical names with counts.
- JD-specific: `rank_jd_skills(jd_text)` adds `mentions` (count) and `importance`:
  - `high`: appears in a sentence with "required", "must", "mandatory", "essential", or mentioned 3+ times
  - `low`: appears in a sentence with "preferred", "nice to have", "bonus", "plus", "good to have"
  - otherwise `medium`

Limitation to document: skills not in the database are not detected.

#### 10.5.3 `matcher_tfidf.py` — Objective 2 (baseline)

- scikit-learn `TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True, stop_words=<NLTK English>, token_pattern=r"(?u)\b[\w+#.]+\b")`.
- Fit on a reference corpus (all texts in `eval/resumes` and `eval/jds`, saved with `joblib`). If the corpus is missing, fit on the pair itself.
- `score = min(100, cosine(resume_vec, jd_vec) / TFIDF_CEILING * 100)`.
- Raw cosine values for resume vs JD are naturally low, so the ceiling scales them to something readable. Tune with the labelled data (Section 12).

#### 10.5.4 `matcher_embed.py` — Objective 3

- Load `all-MiniLM-L6-v2` **once at startup** (not per request). Run on CPU.
- This model only reads about 256 word pieces, so split both texts into chunks of ~100–150 words (prefer sentence/bullet boundaries, small overlap) and embed every chunk.
- **Coverage score:** for each JD chunk, take the highest cosine against any resume chunk; the score is the mean of those maxima. This answers "is every requirement in the job description covered somewhere in the resume?"
- `score = min(100, coverage / EMBED_CEILING * 100)`.
- **Explanation:** keep each JD chunk's best-matching resume chunk. Return the 3 highest as `strong_matches` and the 3 lowest as `weak_spots`. This is the "explains the score" feature from the literature review.
- Mean-pooled whole-document cosine is allowed as a secondary experiment, but not the default.

#### 10.5.5 `gap.py` — Objective 4

- `missing = JD skills − resume skills`; `matched = intersection`.
- Sort missing by importance (high → low), then by mentions.
- Attach `resources` from `learning_links` (up to 3 per skill, beginner first).
- `coverage_percent = matched / total JD skills * 100`.
- `by_category`: for each category present in the JD, `required` and `matched` counts (feeds the radar chart).

- If the JD contains no known skills, return empty lists plus a warning.

#### 10.5.6 `ats.py` — Objective 5

Return a list of checks, each `{id, title, status: pass|warn|fail, detail, fix}`. **At least 8**; use these 10:

| id | Check | How |
|---|---|---|
| `text_extractable` | Text can be extracted | Enough characters per page; fail if scanned image |
| `multi_column` | Single-column layout | Cluster word x-positions; warn if 2+ strong columns |
| `tables` | No tables | `page.find_tables()`; warn if found |
| `images` | No images holding text | `page.images`; warn if large image areas |
| `section_headings` | Standard section headings | Look for Experience, Education, Skills, Projects, Summary/Objective; warn for each missing |
| `contact_info` | Email and phone present | Regex |
| `length` | Length is 1–2 pages | Warn at 3+ pages |
| `fonts` | Few fonts | Warn when more than 3 font families |
| `special_chars` | Few unusual symbols | Warn when decorative-symbol ratio is high |
| `bullets` | Plain bullets | Warn on non-standard bullet glyphs or icon fonts |

`ats.score = round(100 * passed / total)` where `warn` counts as half. Every `warn` / `fail` must have a short, actionable `fix` message ("Replace the table with plain lines of text.").

#### 10.5.7 `pipeline.py` (orchestrator)

1. Validate input (done in the router; pipeline assumes valid).
2. Parse resume → `ParsedResume`; clean JD.
3. Extract skills for both.
4. Run matcher(s) according to `method`. Run independent work concurrently where it helps (e.g. ATS on a thread).
5. Gap analysis, then ATS, then assemble the response.
6. Record timings per stage and return `processing_ms`. Target: **≤ 5 seconds total** on a normal laptop (Objective 5).

Label from score: `≥ LABEL_GOOD_MIN` → "Good fit"; `≥ LABEL_AVG_MIN` → "Average fit"; else "Low fit".

## 11. Quality, performance and security requirements

- Result in ≤ 5 s for a 2-page resume and a ~400-word JD (after warm-up).
- Load models at startup, never per request. Expose `model_loaded` on `/api/health`.
- Process the uploaded file in memory. Do not write it to disk (or delete the temp file in a `finally` block). Do not store, cache or log resume/JD content; log only request id, sizes and timings.
- Validate the PDF by magic bytes, size and page count. Reject anything else.
- CORS limited to `ALLOWED_ORIGINS`. Simple per-IP rate limit (e.g. 20 requests/minute) is a nice-to-have.
- Never return stack traces to the client.
- Show a "responsible use" note in the UI: the fit score is an estimate to help applicants improve; it must not be used to make hiring decisions about people.

## 12. Evaluation harness (Objectives 2 and 3)

**Dataset** (`eval/pairs.json`), minimum **10 pairs, aim for 20**, balanced across labels:

```
[{"id": "p01", "resume_file": "resumes/r01.pdf", "jd_file": "jds/j03.txt",
  "label": "good", "notes": "Strong React + FastAPI match"}]
```

- `label` ∈ `good | average | poor`, assigned by the team by hand. Best practice: 2 members label independently, then resolve disagreements. Report how many disagreed.
- Use your own resumes, consented friends' resumes, or synthetic ones. Do not use anyone's private resume without permission.

`eval/evaluate.py`

1. Run the TF-IDF and embedding scores for every pair (calling the same service functions the API uses).
2. Convert each score to a predicted label with the thresholds in `config.py`.
3. Metrics per method: **agreement %** (`predicted == hand label`), confusion matrix, Spearman correlation between score and ordinal label (`poor=0, average=1, good=2`), and mean score per label.
4. Write `eval/report.json`:

```
{"generated_at": "2026-11-20T10:00:00Z", "pairs": 20,
 "thresholds": {"good_min": 65, "average_min": 40},
 "methods": {
   "tfidf":      {"agreement_percent": 55.0, "spearman": 0.52, "confusion": [[...]]},
   "embeddings": {"agreement_percent": 75.0, "spearman": 0.74, "confusion": [[...]]}},
 "rows": [{"id": "p01", "label": "good", "tfidf_score": 44.1, "embedding_score": 78.0,
           "tfidf_pred": "average", "embedding_pred": "good"}]}
```

5. Print a short summary and whether the Objective 3 target (≥ 70%) is met.

**Honesty rules for the report:** with a small dataset, tuning the thresholds on the same pairs makes results look better than they are. Report results with the default thresholds first, and mention that any tuning was done on the same pairs. If embeddings do not beat TF-IDF, report that. It is still a valid result.

# 13. Frontend specification

## 13.1 Stack and setup

React (Vite, plain JavaScript/JSX), `react-router-dom`, `axios`, `chart.js` with `react-chartjs-2`. Styling: plain CSS with CSS variables (no UI framework). Env var `VITE_API_BASE_URL` (default `http://localhost:8000`).

## 13.2 Design language

Use the same look as the review presentations so the demo and slides feel like one product.

| Token | Value | Use |
|---|---|---|
| `--midnight` | `#21295C` | Header, footer, headings |
| `--primary` | `#065A82` | Primary buttons, links, gauge |
| `--teal` | `#1C7293` | Secondary elements, chart series |
| `--mint` | `#5FB9C9` | Highlights, chips |
| `--coral` | `#C65B3C` | Missing skills, fail states |
| `--success` | `#2F855A` | Matched skills, pass |
| `--warning` | `#B7791F` | Warn states |
| `--bg` | `#F3F7FA` | Page background |
| `--surface` | `#FFFFFF` | Cards |
| `--text` | `#1A1A2E` | Body text |
| `--muted` | `#5B6B79` | Secondary text |

- Fonts: Inter for body and UI, Source Serif 4 for page headings (Google Fonts).
- Cards with 12px radius and a soft shadow; 8px spacing grid; max content width 1120px.
- Color must never be the only signal: pair every pass/warn/fail with an icon and a word.
- Mobile-first. Breakpoints at 640px and 1024px. On mobile the results sections stack in one column.
- WCAG AA contrast, visible focus rings, full keyboard use, `aria-live="polite"` for status messages, alt text/labels on all controls.

## 13.3 Routes and pages

**Header (all pages):** logo text "Resume Fit Analyzer" + nav: Analyzer · How it works · Evaluation · About. **Footer:** project name, "Mini Project · TY Computer Engineering", team names, responsible-use note, privacy note.

**Page 1 — Analyzer ( `/` ) — the main page**

*Top section (before analysis):*

- Heading: "Know your fit before you apply." Subheading: "Upload your resume, paste a job description, and see your fit score, missing skills and a plan to close the gap."
- **Input card** with two parts side by side (stacked on mobile):
  - **Resume upload:** drag-and-drop zone + "Browse" button, PDF only. After selecting: show file name, size and a remove button. Inline errors: wrong type, over 5 MB.
  - **Job description:** textarea (min height ~220px), live character counter (min 50, max 10,000), "Clear" button, and **"Try a sample"** chips (Frontend Developer, Backend Developer, Data Analyst) that fill the textarea from `data/sampleJobs.js`.
- **Analyze** button, disabled until both inputs are valid (the reason is shown next to it when disabled).
- Privacy line: "Your resume is processed in memory and never stored."

*Loading state:* replace the button with a progress panel showing four stages in order: "Reading your resume → Finding skills → Matching with the job → Checking ATS format". Advance the stages on a timer and finish when the response arrives. If the request takes longer than 8 seconds, show "The server is waking up. The first request can take up to a minute."

*Results (appear below the input card, page scrolls to them; the input card collapses to a summary bar "Resume.pdf · Job description (412 words) · Edit"):*

1. **Fit summary card**
   - Large circular gauge (Chart.js doughnut) with the primary score (e.g. 72%) and the label ("Good fit").
   - Two small chips: "TF-IDF 41%" and "Embeddings 72%", each with an info tooltip explaining the method in one sentence (text from Section 7.1). The primary method is marked.
   - "Analyzed in 2.3 s".
2. **Skills card**
   - Coverage bar: "You match 5 of 8 required skills (62%)".
   - Two groups of chips: **Matched** (green, check icon) and **Missing** (coral, sorted by importance, each with an importance badge: High / Medium / Low).
3. **Skill coverage by category** — Chart.js **radar** chart using `by_category` (matched ÷ required per category). If fewer than 3 categories, use a horizontal bar chart instead.
4. **Method comparison** — small Chart.js **bar** chart: TF-IDF score vs embeddings score for this pair, with one sentence of explanation.
5. **Learning roadmap** — one card per missing skill (high importance first): skill name, importance badge, "Mentioned N times in the job description", and 1–3 resource links (title, provider, type icon, level, "Free" tag). Links open in a new tab with `rel="noopener noreferrer"`. Each card has a "Mark as done" checkbox (state kept only in the browser, e.g. localStorage with try/catch). Above the list: "Copy roadmap as text" and "Print / Save as PDF" buttons (a print stylesheet that hides the input card, nav and buttons).
6. **ATS check** — overall ATS score ring + "7 of 10 checks passed". List of all checks with a pass/warn/fail icon and title; warn/fail rows expand to show the detail and the fix.
7. **Why this score?** — two columns: "Strongest matches" and "Weakest spots". Each item shows the job-description snippet next to the closest resume snippet and a similarity bar. A short note: "Weak spots are parts of the job description that nothing in your resume covers well."
8. **Actions:** "Analyze another resume", "Edit job description and re-run".

*Empty and error states:*

- No missing skills: celebratory message "No missing skills found from our skills list."
- No skills detected in JD: show a warning that the JD may be too short or uses skills outside the database.
- API errors: a dismissible banner with the server's message and a "Try again" button; map error codes to friendly text (e.g. `unreadable_pdf` → "We couldn't read text from this PDF. It may be a scanned image. Export it as a text PDF from Word or Google Docs and try again.").
- Network failure: "Can't reach the server."

**Page 2 — How it works ( `/how-it-works` )**

- The 6-step pipeline as a horizontal stepper (stacked on mobile): 1 Upload → 2 Extract & clean → 3 Identify skills → 4 Convert text to vectors → 5 Compute match score → 6 Analyze gap & show results, each with a one-line explanation (use the wording in Section 8).
- "TF-IDF vs sentence embeddings" explainer with the "built websites using React" vs "frontend developer experience" example (Section 7.1).
- The architecture diagram image ( `docs/architecture.svg` ).

**Page 3 — Evaluation ( `/evaluation` )**

- Calls `GET /api/evaluation` . If `no_report` , show "Evaluation has not been run yet."
- Summary tiles: number of pairs, TF-IDF agreement %, embeddings agreement %, and whether the 70% target is met (green/red with text).
- Bar chart comparing the two agreement percentages with a dashed target line at 70%.
- Table of rows: pair id, hand label, TF-IDF score and predicted label, embedding score and predicted label, with correct predictions marked ✓ and wrong ones ✗.
- Confusion matrices for both methods (3×3 grids).
- Footnote with the honesty note from Section 12.

**Page 4 — About ( `/about` )**

- Problem statement and purpose (Sections 1–2).
- Objectives table with the weeks (Section 6.1).
- Literature review summary table (Section 4) with the links.
- Tech stack table (Section 7).
- Team members and guide (placeholders), plus "Future ideas" (the out-of-scope list in Section 3).

### 13.4 Components (suggested)

`Header` , `Footer` , `FileDropzone` , `JobDescriptionInput` , `SampleChips` , `AnalyzeButton` , `ProgressStages` , `ErrorBanner` , `ScoreGauge` , `MethodChips` , `SkillChips` , `CoverageBar` , `CategoryRadar` , `MethodBarChart` , `RoadmapCard` , `AtsPanel` , `ExplanationPanel` , `EvalSummary` , `ConfusionMatrix` . Keep API calls in `src/api.js` only (one function per endpoint) and shape handling in one place.

### 13.5 Sample data

`src/data/sampleJobs.js` with 3 realistic job descriptions (~150–250 words each: Frontend Developer, Backend Developer, Data Analyst). Also put 2 sample resume PDFs in `backend/tests/fixtures/` and mention in the README that people can use them to try the app.

## 14. Testing

Backend ( `pytest` ), each with a clear pass criterion:

| Area | Tests |
|---|---|
| Parser | ≥ 10 fixture PDFs (single-column, two-column, with tables, with images, a scanned image, a 6-page PDF, a corrupt file renamed `.pdf` ); **at least 9 of 10 readable ones extract correctly** (Objective 1). Scanned/corrupt files raise the right error. |
| Skills | Alias normalisation ( `js` , `reactjs` , `postgres` ), multi-word skills, ambiguous `c` / `r` / `go` , importance detection (required vs preferred). |
| TF-IDF | Identical texts score near 100; unrelated texts score low; scores stay within 0–100. |
| Embeddings | Same ordering expectations; "built websites using React" vs "frontend developer experience" scores higher than vs "cooking recipes". Handles texts longer than the model limit via chunking. |
| Gap | Missing = JD − resume; resources attached; empty cases. |
| ATS | Each of the 10 checks has a pass fixture and a fail fixture. |
| API | Happy path; every error code in Section 10.3; response matches the schema; **no content persisted** (check the temp dir/DB after a request). |
| Performance | End-to-end under 5 s on the fixture pair (after model warm-up). |

Frontend: manual test checklist (upload, drag-and-drop, wrong file, long file name, empty JD, sample chips, loading stages, every error banner, print view, mobile width 375px, keyboard-only use). Optional: Vitest for helpers, Playwright for one end-to-end run.

## 15. Deployment

- **Frontend → Vercel.** Project root `frontend/` , build `npm run build` , output `dist` , env var `VITE_API_BASE_URL` = backend URL.
- **Backend → Render (Web Service, free tier).**
  - Root `backend/` ; build: `pip install -r requirements.txt && python -m spacy download en_core_web_sm && python -c "import nltk; nltk.download('stopwords')" && python scripts/seed_db.py`
  - Start: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
  - Env vars: `ALLOWED_ORIGINS` (the Vercel URL), `PRIMARY_METHOD` , etc.
  - Pre-download the embedding model during the build so it isn't fetched on first request.
- **Known free-tier risks (check current limits before the demo):**

    - Free instances usually have about 512 MB RAM. PyTorch + sentence-transformers may exceed that. First try, in this order: CPU-only PyTorch, then loading the model lazily. Only if that still fails, ask the team before switching to an ONNX-based embedding library such as `fastembed` or hosting the backend on Hugging Face Spaces. Both would change the stack shown on the slides, so the slides (Design & Technology Stack) must be updated too.
    - Free instances sleep after inactivity, so the first request can take up to a minute.
    - Free disks can be wiped. That is fine here because `app.db` is rebuilt from `skills_seed.json` at build time and nothing user-related is stored.
- Add a GitHub Actions workflow (optional) that runs `pytest` on every push.

## 16. Privacy and responsible use (must be visible in the app)

- Resume and job description are processed in memory and discarded. State this on the Analyzer page.
- No accounts, no cookies needed, no analytics that capture content.
- Personal identifiers are stripped before scoring (Section 10.5.1).
- The score is an estimate for self-improvement and is not a hiring decision.
- Mention limitations in the About page: English only, PDF only, skills outside the database are missed, scores depend on the quality of the text extracted.

## 17. Build order for the agent (with acceptance checks)

Do these in order. After each step run the tests and show the result before moving on. The week tags match the objectives table.

| Step | Build | Acceptance check |
|---|---|---|
| 0 | Repo scaffold: backend (FastAPI hello + `/api/health`), frontend (Vite + routes + header/footer + design tokens), `.gitignore`, README with run instructions | Both run locally; frontend can call `/api/health` |
| 1 | `parser.py` + tests + fixtures **(Objective 1, Week 3)** | 9 of 10 fixture PDFs extract correctly; scanned PDF raises `unreadable_pdf` |
| 2 | SQLite schema, `skills_seed.json` (100+ skills, links), `seed_db.py`, `check_links.py` | ≥ 100 skills, every skill has ≥ 1 link, link checker passes |
| 3 | `skills.py` (extraction, aliases, JD importance) | Skills tests pass |
| 4 | `matcher_tfidf.py` + `POST /api/analyze` v0 returning TF-IDF score and skills **(Objective 2, Week 5)** | Scores returned for 10+ labelled pairs; API tests pass |
| 5 | Evaluation dataset + `evaluate.py` (TF-IDF only at first) | `report.json` generated |
| 6 | `matcher_embed.py` + explanation snippets; evaluate both methods **(Objective 3, Week 7)** | Report shows both methods; embeddings agreement ≥ 70% or an honest note if not |

| Step | Build | Acceptance check |
|---|---|---|
| 7 | `gap.py` + roadmap in the response **(Objective 4, Week 8)** | Missing skills carry links; by-category counts correct |
| 8 | `ats.py` with 10 checks | Pass and fail fixtures for each check |
| 9 | Frontend Analyzer page complete (all result sections, loading, errors, print view) | Full flow works against the local backend; mobile layout OK |
| 10 | Frontend How it works, Evaluation, About pages | Evaluation page renders `report.json` |
| 11 | Integration, performance pass, error-code mapping, accessibility pass **(Objective 5, Week 11)** | End-to-end ≤ 5 s; all error states handled; keyboard-only run works |
| 12 | Deploy (Section 15), warm-up check, README, final demo script | Public link works from a phone |

Suggested team split (3 people): **A** — NLP/ML (parser, skills, matchers, evaluation); **B** — frontend (all pages and charts); **C** — API, database and seed data, ATS, deployment, testing, documentation. Review with the guide after each step (the Review stage of the methodology).

## 18. Demo script (for reviews)

1. Before the demo, open `/api/health` to wake the server.
2. Show the Analyzer with a sample resume and the "Backend Developer" sample job. Point out the score, the two method chips, missing skills and the roadmap links.
3. Open the ATS panel with a deliberately bad resume (two columns + table) to show warnings and fixes.
4. Open "Why this score?" to show the explanation.
5. Open the Evaluation page: TF-IDF vs embeddings agreement with hand labels, against the 70% target.
6. Open How it works, then the architecture diagram.

## 19. Definition of done

- All five objectives meet their measures (Section 6.1) and the evidence is visible in tests or the Evaluation page.
- Analyzer, How it works, Evaluation and About pages work on desktop and mobile.
- ≥ 8 ATS checks implemented (10 specified).
- 100+ skills, each with at least 1 working free link.
- Result in ≤ 5 seconds after warm-up.
- No resume or job description content is stored or logged.
- Deployed on free tiers with a working public link; README explains setup, environment variables and how to rerun the evaluation.

## 20. Glossary

- **ATS:** Applicant Tracking System. Software companies use to scan and filter resumes.
- **TF-IDF:** Term Frequency–Inverse Document Frequency. Scores words by how often they appear in a document and how rare they are elsewhere.
- **Cosine similarity:** Measures how similar two lists of numbers (vectors) are by the angle between them. 1 = same direction, 0 = unrelated.
- **Sentence embedding:** A list of numbers that captures a sentence's meaning, produced by a pre-trained model.
- **Skill gap:** Skills the job asks for that the resume does not show.
- **AUC:** Area under the ROC curve. A score from 0.5 (guessing) to 1.0 (perfect) for how well a model separates good matches from bad ones.
- **Agreement %:** Share of test pairs where the system's predicted label equals the team's hand label.